Analysis and Comparison of Policies

Qussai Albalkhi

Table of Contents

Overview 3
Implementation Summary 3
allocate_frame(): 3
find_physpage(): 3
Per-Algorithm Implementation Details 4
Testing Methodology 4
Results Summary 5
Policy Comparisons with Minimal Traces 6
FIFO vs LRU: 6
LRU vs CLOCK 7
LRU vs ARC 8
Why Each Policy Wins in Its Favourable Race 9
Recommendations 9
Notes on Using Traces 10
Edge-Case and Robustness Verification 10
Performance Interpretation 10
Conclusion 11

Overview

The assignment involved designing and implementing a virtual memory simulator with the capability to simulate demand paging and execute different page replacement algorithms such as FIFO, LRU, CLOCK and ARC algorithms in addition to others. The simulator handles translation between the virtual and physical addresses, page faults, and interaction between the physical memory and the swap space to offer information on the performance of the algorithms in terms of hits and misses, as well as clean and dirty evictions.

Implementation Summary

allocate_frame():

The implementation involved finishing off the two essential tasks: the implementation of functions allocate_frame() and find_physpage(), in addition to implementing the individual algorithms in separate files. In the function allocate_frame(), the simulator checks if there are any free frames in memory if there aren't, it calls the eviction function defined in the current active algorithm to identify the page to be removed. The page to be removed, whether clean or dirty, has to go to the swap space through swap_pageout() to ensure that it has a valid swap position every time. The page table is updated to mark the page not valid, not referred to, not dirty, with the on-swap flag on, and stores the corresponding new swap position.

find_physpage():

The function find_physpage() manages page faults. When the page being referred to is in swappable space, the simulator checks if it needs to be swapped in, and if so, swaps it back in with the help of swap_pagein(). If not, it allocates a new page with the help of init_frame(). It marks the page valid and referred to, with the dirty bit set if necessary. The ref_func(), related to the algorithm, is also called after every successful access to modify algorithm data appropriately according to page recency or usage.

Per-Algorithm Implementation Details

Each replacement policy is defined in a separate file (fifo.c, lru.c, clock.c, arc.c).

Figure 1: Time and Space Complexity for Different Policies

Algorithm	Time Complexity	Space Complexity	Notes
FIFO	O(1)	O(1)	Implemented with a single integer counter for the next eviction (circular index).
LRU	O(1)	O(M)	Doubly linked list with per-frame hash mapping for constant-time updates.
CLOCK	O(M) worst case	O(1)	Circular scan with one reference bit per frame.
ARC	O(1) amortized	O(M)	Two adaptive lists (recency and frequency) with ghost tracking.

Testing Methodology

Testing was designed to verify both correctness and robustness under varying conditions. All five algorithms were tested using a mix of provided and custom traces:

Figure 2: Algorithms Run on Different Traces

Trace	Algorithms Tested	Description
page-matmul.ref	FIFO, LRU, CLOCK, ARC	High temporal locality; includes write operations
page-blocked.ref	FIFO	Blocked access with improved spatial locality
page-simpleloop.ref	FIFO, LRU, CLOCK, ARC	Sequential loop pattern
page-repeatloop.ref	FIFO, LRU, ARC	Repetitive scanning behavior
rand.trace *(custom)*	ARC	Randomized, low-locality access pattern
page-mixed.ref *(custom)*	ARC	Multi-phase mixed workload

Each trace was tested with varying physical (-m) and swap (-s) sizes to observe scalability and correctness.

Results Summary

For the matrix multiplication application with average memory space (32 frames, 64 swap slots), the hit ratios were close to 96–97 percent for all algorithms tested on the simulator. The LRU and ARC policies consistently showed higher hit ratios, albeit with a small difference, in comparison with the FIFO and CLOCK strategies on the average memory space application of matrix multiplication tasks with repeated access patterns in the real implementation.

When the memory size was varied, the result was as expected. When there were only four frames in memory, the hit ratio was less than 10 percent, indicating intense page replacement activity in memory. When memory was increased to 32 frames, the hit ratio surpassed 97 percent, and after that point, the progress was marginal because the working set fit well in the physical memory.

The trace with random access gave the ARC algorithm very little reuse, so it resulted in the ARC algorithm getting only about 7% hits and above 90% misses in the trace. While the performance was expected to be low due to random access, the simulator was stable, and it calculated all events correctly with no invalid swaps.

Figure 1: Algorithm Comparison (Matrix Multiply Trace, m = 32, s = 64)

Algorithm	**Hit Rate (%)**	**Miss Rate (%)**	**Clean Evictions**	**Dirty Evictions**
FIFO	96.50	3.50	46	18
LRU	**97.12**	2.88	34	13
CLOCK	96.72	3.28	42	16
ARC	**97.15**	2.85	33	13

Figure 2: Memory Size Scaling (Matrix Multiply, LRU)

Physical Frames	**Hit Rate (%)**	**Miss Rate (%)**
4	9.08	90.92
32	97.12	2.88
2048	97.77	2.23

Figure 3: Random Access Trace (rand.trace, ARC, m = 16, s = 1024)

Hits	**Misses**	**Clean Evictions**	**Dirty Evictions**	**Hit Rate (%)**
70	930	914	0	7.00

Policy Comparisons with Minimal Traces

FIFO vs LRU:

To show where LRU is superior to FIFO, consider a short working-set pattern that repeatedly reuses the most recent pages. With a frame budget of two, the trace

```
L 0x0000
L 0x1000
L 0x2000
L 0x0000
L 0x1000
L 0x2000
```

creates a tight “recently used” cycle. LRU keeps the last two pages and suffers fewer misses because it always discards the least recent page, which is precisely the one not needed immediately. FIFO, however, may evict a page purely because it is oldest, even if it is about to be referenced again, which increases its miss count. In a longer program exhibiting the same recency-heavy loop, the gap between FIFO and LRU widens: LRU continues to retain the immediately useful pages, while FIFO keeps discarding them as they age out.

A contrasting case where FIFO can do better than LRU is a short sequence in which the very recent page is a *red herring* but an older resident is about to be reused. With two frames, the concrete trace

```
L 0x0000
L 0x0000
L 0x3000
L 0x2000
L 0x3000
L 0x0000
L 0x2000
L 0x0000
```

causes LRU to evict the older, still-useful page more often because it clings to the most recently touched page even when that page will not reappear immediately. FIFO, by ignoring recency and evicting strictly by arrival order, avoids one of those premature evictions on this particular pattern and finishes with fewer misses. While LRU is generally stronger, carefully constructed patterns like this show that FIFO can occasionally win by not “overfitting” to the most recent touch.

LRU vs CLOCK

Because CLOCK approximates LRU with a single reference bit, they often behave similarly, but small traces reveal the difference. In a pattern where the last-used page truly *is* the best predictor, LRU has the edge. With three frames, the trace

```
L 0x3000
L 0x2000
L 0x1000
L 0x1000
L 0x3000
L 0x1000
L 0x4000
L 0x1000
L 0x3000
L 0x3000
```

favours LRU because it evicts the page that has been unused for the longest time. CLOCK can lag behind when many pages have their reference bit set; it must sweep and clear bits before finding a victim, and in doing so it may evict a page that LRU would have kept, yielding one extra miss on this short sequence. Over a larger run with the same "true LRU" character, LRU's advantage persists modestly.

There are also patterns where CLOCK beats LRU by not being as reactive to a single touch. With three frames, the trace

```
L 0x4000
L 0x2000
L 0x4000
L 0x2000
L 0x4000
L 0x1000
L 0x3000
L 0x2000
L 0x2000
L 0x4000
```

can give CLOCK a miss fewer than LRU. Here, intermittent toggling keeps reference bits set on the truly useful pages, so CLOCK's second-chance sweep protects them when a new page arrives. LRU, by contrast, may evict a page that was just slightly less recent—even though that page remains part of the active working set—and then pay an extra miss when it quickly reappears. On longer runs with the same toggling behaviour, CLOCK's conservatism continues to avoid a handful of LRU's "almost-recent" mistakes.

LRU vs ARC

ARC adapts between recency and frequency using its T1/T2 and ghost lists; as a result, it can decisively outperform LRU on “scan-plus-hot-item” patterns where pure recency fails. With two frames, the trace

```
L 0xA000
L 0x1000
L 0x2000
L 0x3000
L 0x4000
L 0x5000
L 0xA000
L 0x6000
L 0x7000
L 0x8000
L 0x9000
L 0xA000
```

interleaves a long one-time stream with a truly frequent page at 0xA000. LRU suffers because the stream continuously pushes the hot page out; each return to 0xA000 faults. ARC, however, learns from repeated “ghost” hits on the hot page and shifts capacity toward its frequent set, protecting 0xA000 in T2 so that subsequent references to it become hits. In a much longer trace with the same structure—many unique stream pages with sporadic returns to a few genuinely frequent pages—the advantage of ARC over LRU grows steadily as ARC’s adaptation locks in the protection for those frequent pages.

There are also short, purely recency-driven patterns where LRU can beat ARC due to ARC’s adaptation overhead at startup. Consider, with two frames, the compact recency loop

```
L 0x0000
L 0x1000
L 0x0000
L 0x1000
L 0x0000
L 0x1000
```

in which no page is “frequent” in the sense of appearing beyond its immediate alternation. LRU is optimal here and converges immediately. ARC starts with a balanced split between recency and frequency and may briefly over-reserve space for its frequent list before its parameter p swings fully toward recency; that transient can cost ARC a miss that LRU avoids on such tiny sequences. Over long runs of the same pure alternation, ARC converges and essentially ties LRU, but on very short bursts LRU can finish ahead by a nose.

Why Each Policy Wins in Its Favourable Race

In the FIFO-friendly sequence, neglecting recency avoids fooling FIFO by a recently accessed page that won't be needed for a while, while LRU's dogmatic adherence to “last touch” forces an unnecessary eviction, while in the LRU-friendly loop, it is just the reverse—recent is best, and it is only because of LRU's stack policy that it retains just the pages the loop needs next, while FIFO would throw them away because they're old.

Between LRU and CLOCK, it is CLOCK’s single reference bit that makes the difference. When there are lots of pages that have been recently accessed, CLOCK has to flip bits to mark pages, potentially discarding a useful page even if “true-LRU” would not discard it, so in “true-LRU” access patterns, it is true-LRU that wins. On the other hand, if correct pages are maintained marked and incorrect ones lose their bit while pages are being intermittently accessed, CLOCK’s “second chance” cleanup just barely beats out true-LRU.

The difference between LRU and ARC in terms of ghost lists is in the details of ARC that implement the realization “the thing we just evicted was important” with B1/B2 to adjust p to give higher priority to frequently accessed pages: “so the hot item is retained and hits are made to it.” In small, purely recency-based cycles, ARC only needs to “adapt to the split” after a couple of “touches” so that it can “truly” favor recency, giving LRU “a one-round lead before ARC can catch back up.”

Recommendations

For trace caches in tight loops and working sets where the last used pages are the best predictors for the next access, LRU should definitely be the preferred choice because it always has the lowest misses on pure recency traces and does well if the working set fits in the memory. CLOCK is the preferred choice if implementation overhead counts because it very closely follows the misses of LRU with less bookkeeping and might even outperform it on short, noisier traces because of the benefit of the second-chance algorithm.
ARC is the preferred choice if there are traces with mixed workloads in terms of one-time traces and traces where there are only a very few pages that access more frequently because it can protect the frequent pages from being polluted by one-time traces in a manner that LRU and CLOCK fail to utilize properly. FIFO only has any right to be used in traces where implementation ease counts significantly because it fails to utilize traces properly in terms of pure recency because it doesn't take it into account and might end up with more faults in general because of it, though it can avoid the over-eagerness of traces in one specific trace in which it can outperform traces in terms of misses because it doesn't utilize the information that it can increasingly utilize in terms of misses in longer traces.
Because it only keeps track of the last dozen or so traces, it follows the same traces with increasingly multiple hits, and that makes it follow the same traces with increasingly multiple hits because it doesn't matter if there are only traces in terms of dozens. Traces that prefer one policy on the dozen traces tend to prefer it on traces in terms of thousands because traces tend to increasingly follow the same trace, and because it keeps on replaying traces on increasingly multiple hits, the pattern repeats in terms of increasingly multiple hits as trace

Notes on Using Traces

All addresses are spaced by 0x1000 so each token refers to a distinct page in your simulator. You can paste each snippet into a file and run them with your current binary, for example ./sim -f fifo_lru_win.trace -m 2 -s 64 -a fifo and ./sim -f fifo_lru_win.trace -m 2 -s 64 -a lru, to reproduce the differences described above. If you want me to package these four comparisons as separate .trace files with suggested -m/-s/-a commands, say the word and I'll drop them in ready to run.

Edge-Case and Robustness Verification

Additional tests confirmed that the simulator works properly in extreme cases by additional tests. The process ran perfectly with only one frame in the pool, with the system experiencing perpetual eviction cycles yet being stable with correct statistic information. Narrowing the swap space to provoke a "swap full" notice confirmed that the simulator prints a correct notice before gracefully halting without crashing. Increasing the swap space to a few thousand slots allowed the execution of traces with an extended length and increasing the physical memory to 2048 frames resulted in very few page faults, thus validating the correct simulator operation for the zero-eviction context because page faults became just page requests in that situation. With the help of the trace of the matrix multiplication application, the dirty page information was checked to be correct because it showed the clean/dirty ratio of the total evictions to be correct regarding the reference solution. The random and mixed traces further validated that the simulator operates properly with workloads of locality and hybrid types because they operated smoothly in such cases. Victim-page selection followed proper algorithmic rules as shown by debug-mode output.

Performance Interpretation

Overall, the ARC and LRU algorithms were the best-performing ones in every environment. ARC was always able to outperform LRU in hit ratios by a marginal difference of just a few hundredths of a percent because it can automatically adjust between the two extremes: page reuse and page recency by varying the parameter p in accordance with the nature of the access pattern—increasing it if there is repeated reuse of the same pages, and vice versa.

CLOCK was only slightly less than LRU in performance, thus validating the operation of the reference bit to estimate recurrence with smaller overhead. FIFO maintained the lowest values for hit ratios because it failed to take into account fresh versus stale pages, while RAND was primarily used in the control group, with performance being consistently bad because it did not factor in any notions of locality.

For each of the algorithms, it was noticed that the increase in the memory size was non-linear, with the hit ratio rising rapidly at the start and reaching saturation after the working set was entirely held in memory. The observed pattern in the algorithms is remarkably close to real-world processes, ensuring that the simulators capture the essential aspects of page replacement methodology properly. The variations in the number of dirty and clean page evictions were also consistent with expected write-intensive and read-intensive periods in the traces.

Conclusion

The comparison between FIFO, LRU, CLOCK, and ARC serves to illustrate that vastly different replacement strategies can have a profound effect on overall performance, depending on the access pattern. LRU and ARC consistently yield the highest hit rates because of their sensitivity to temporal locality, while FIFO's inflexible order of replacement often leads to unnecessary evictions. CLOCK closely approximates LRU but introduces slight variations caused by its single-bit reference mechanism and, therefore, represents a simpler yet efficient compromise. As the traces that were developed revealed, FIFO can sometimes outperform LRU in patterns where very recent pages are misleadingly accessed once, while LRU comes out best for cyclic recency-driven workloads. While less precise, CLOCK achieves near-LRU results with less overhead; ARC dynamically adapts to changing access patterns, hence beating the others when the workload mixes sequential scans with recurring hot pages.

In practice, FIFO is adequate for applications where simplicity and predictability are more important than an optimal hit ratio. LRU is ideal in systems with moderate amounts of memory and strong locality, while CLOCK serves as a scalable approximation of LRU on systems in hardware or operating systems where metadata per-page must remain minimal. ARC represents the best general-purpose algorithm, performing consistently well in a wide range of workloads by adaptively balancing recency and frequency. Taken all together, these experiments confirm that no policy is universally optimal; the best choice depends strongly on workload characteristics. Understanding this behavior does have practical value, allowing a developer to choose the most appropriate replacement strategy for a particular context and predict how such patterns would scale to larger systems.